

МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»



НГТУ НЭТИ | Факультет прикладной математики и информатики

Кафедра теоретической и прикладной информатики

Лабораторная работа № 1

по дисциплине «Языки программирования и методы трансляции»

ОСНОВНЫЕ ЭЛЕМЕНТЫ ЯЗЫКА АССЕМБЛЕРА

Бригада 7 ПМ-35 МАРЧЕНКО АЛЕКСАНДР

Вариант 7 ПМ-35 РЕПЕЦКИЙ СТАНИСЛАВ

ПМ-35 ПОКУЛЕВСКИЙ ДАНИЛ

Преподаватель КУТУЗОВА ИРИНА АЛЕКСАНДРОВНА

Новосибирск, 2026

1 Цель работы

Изучить и приобрести практические навыки работы с основными командами ассемблера, регистрами, памятью, символьными данными и функциями консольного ввода-вывода Windows.

2 Содержание работы

1. Изучить основные команды языка Ассемблера и функции операционной системы, используемые для консольного ввода и вывода данных.
2. Разработать программу на языке Ассемблера по варианту 7, которая должна:
   - ввести с клавиатуры число x в восьмеричной системе счисления;
   - вывести введённое число в десятичной системе счисления;
   - вычислить значение полинома
   $$y = 11x^2 - 2x + 1;$$
   - вывести полученный результат в двоичной системе счисления;
   - вывести результат в десятичной системе счисления.
3. Все промежуточные результаты вычислений сохранять в памяти. Преобразование чисел для вывода выполнять без использования функции `wsprintfA`.
4. Сопровождать ввод данных и вывод результатов понятными текстовыми сообщениями.
5. Отладить программу и проверить правильность её работы на тестовых примерах.

3 Алгоритм

1. Получить стандартные дескрипторы консольного ввода и вывода.
2. Считать с клавиатуры число x, записанное в восьмеричной системе счисления. Проверить наличие завершающих символов CR и LF и исключить их из дальнейшей обработки.
3. Преобразовать введённую строку в целое число. Для каждой восьмеричной цифры определить её числовое значение и последовательно вычислять
   $$x = 8x + d,$$
   где d — значение очередной цифры. При наличии знака «$-$» изменить знак полученного числа. Некорректные символы считать ошибкой ввода.
4. Сохранить полученное значение x в памяти и вывести его в десятичной системе счисления.
5. Вычислить значение полинома
   $$y = 11x^2 - 2x + 1.$$
   Вычисления выполнить последовательно:
   $$x^2, \qquad 11x^2, \qquad -2x, \qquad 11x^2 - 2x, \qquad 11x^2 - 2x + 1.$$
   Каждый промежуточный результат сохранить в памяти. При выполнении арифметических операций контролировать переполнение.

6. Преобразовать значение y в двоичную систему счисления. Последовательно выполнять логический сдвиг вправо на один бит командой SHR. Вытесненный младший бит попадает во флаг переноса CF; получить его значение командой ADC в предварительно обнулённом регистре. На каждой итерации сохранять сдвинутое число и полученный бит в памяти, преобразовывать бит в символ и записывать его в выходной буфер справа налево. После получения нулевого числа вывести сформированную строку.

7. Сформировать десятичную запись y последовательным делением на 10. На каждой итерации сохранять частное и остаток в памяти, преобразовывать остаток в символ и записывать его в буфер справа налево. После получения нулевого частного вывести сформированную строку.

8. При корректном выполнении завершить программу. При обнаружении ошибочного ввода или переполнения вывести сообщение об ошибке.

4 Использованные программные средства

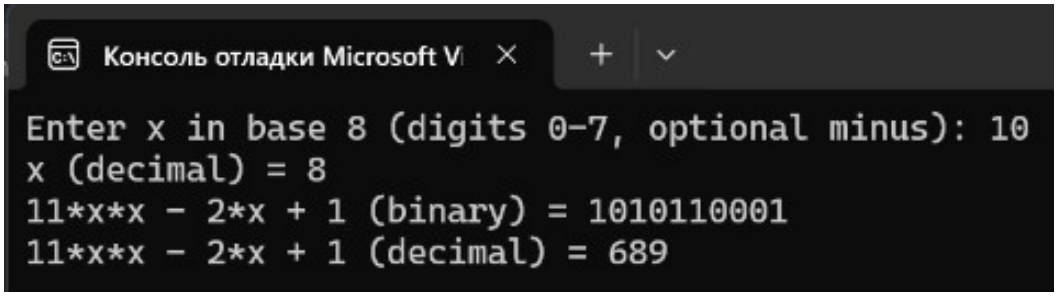
В программе используются следующие директивы, команды, функции и процедуры:

- `.386` — разрешает использование команд процессора 80386.
- `.MODEL FLAT, STDCALL` — задаёт плоскую модель памяти и соглашение вызовов STDCALL.
- `.STACK` — задаёт размер стека программы.
- `.DATA` — определяет начало сегмента данных.
- `.CODE` — определяет начало сегмента кода.
- `DB` — выделяет память размером в один байт.
- `DD` — выделяет память размером в двойное слово (32 бита).
- `DUP` — создаёт последовательность однотипных элементов в памяти.
- `EXTERN` — объявляет внешнюю функцию.
- `PROC, ENDP` — определяют начало и конец процедуры.
- `END` — завершает описание программы.
- `OFFSET` — получает адрес объекта в памяти.
- `BYTE PTR` — указывает, что операнд в памяти имеет размер один байт.
- `MOV` — пересылает данные между регистрами и памятью.
- `MOVZX` — пересылает значение с расширением нулями.
- `ADD` — выполняет сложение.
- `ADC` — выполняет сложение с учётом флага переноса CF; получает вытесненный бит после сдвига.
- `SUB` — выполняет вычитание.

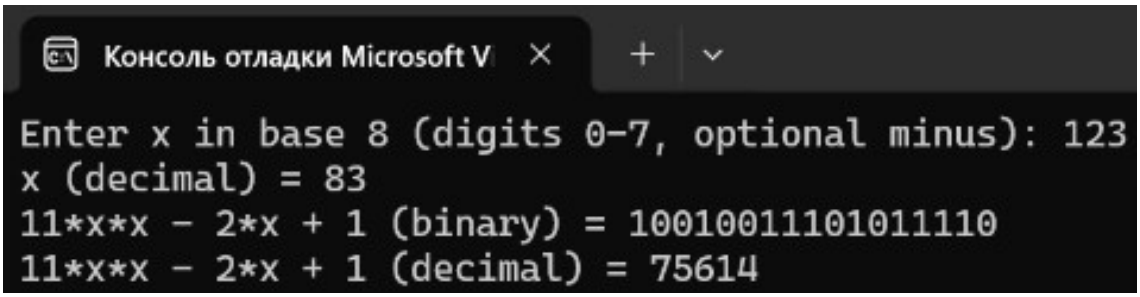
- `IMUL` — выполняет знаковое умножение.
- `DIV` — выполняет беззнаковое деление.
- `SHR` — выполняет логический сдвиг вправо; вытесненный бит сохраняется во флаге переноса CF.
- `NEG` — изменяет знак числа.
- `INC` — увеличивает значение на единицу.
- `DEC` — уменьшает значение на единицу.
- `XOR` — выполняет операцию «исключающее ИЛИ»; используется также для обнуления регистра.
- `CMP` — сравнивает два значения и устанавливает флаги процессора.
- `TEST` — выполняет логическую проверку значения с установкой флагов.
- `JMP` — выполняет безусловный переход.
- `JE, JZ` — выполняют переход при равенстве или нулевом результате.
- `JNE` — выполняет переход при неравенстве.
- `JB, JA, JAE, JGE` — выполняют условные переходы по результатам сравнения.
- `JO` — выполняет переход при возникновении знакового переполнения.
- `PUSH` — помещает значение в стек.
- `CALL` — вызывает процедуру или функцию.
- `RET` — возвращает управление из процедуры.
- `GetStdHandle` — получает стандартный дескриптор ввода или вывода.
- `ReadConsoleA` — считывает строку из консоли.
- `WriteConsoleA` — выводит строку в консоль.
- `ExitProcess` — завершает выполнение программы.
- `PrintText` — выводит нуль-терминированную строку в консоль.
- `ReadOctal` — считывает и преобразует восьмеричное число в целое значение.
- `PrintNumber` — формирует двоичную запись сдвигами, десятичную — делением на 10, и выводит строку.
- `MAIN` — основная процедура программы.

5 Тестирование

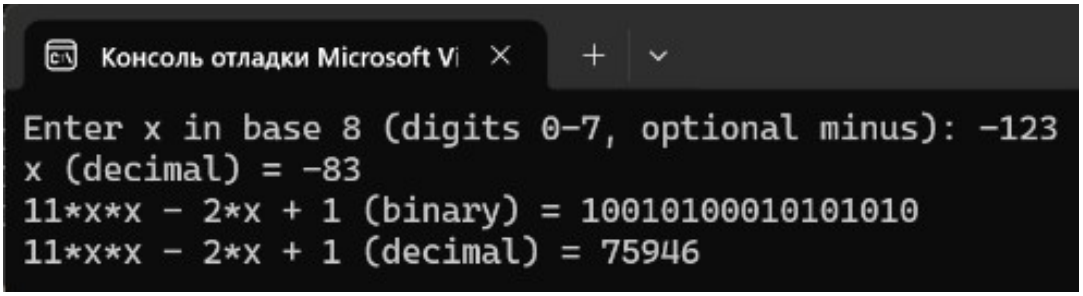
Тест № 1
Назначение: Проверка вычислений на простом входном значении.
Ожидается: $x_{10} = 8$, $y_{10} = 689$.



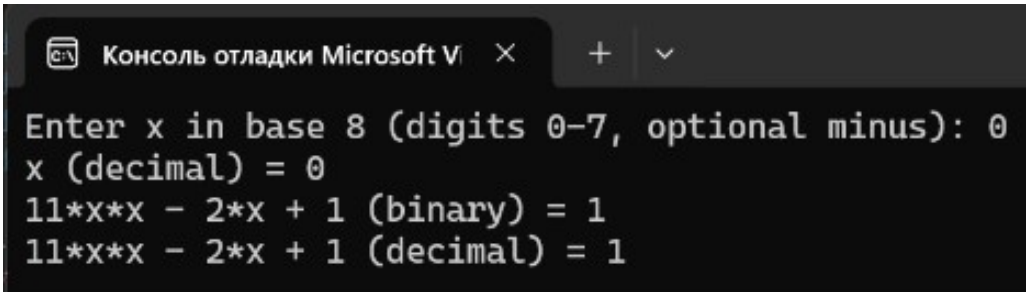
Тест № 2
Назначение: Проверка вычислений на многозначном положительном числе.
Ожидается: $x_{10} = 83$, $y_{10} = 75614$.



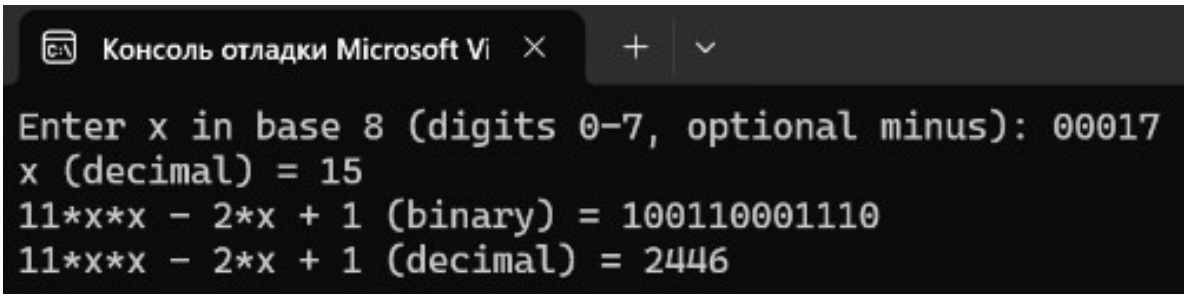
Тест № 3
Назначение: Проверка вычислений на многозначном отрицательном числе.
Ожидается: $x_{10} = -83$, $y_{10} = 75946$.



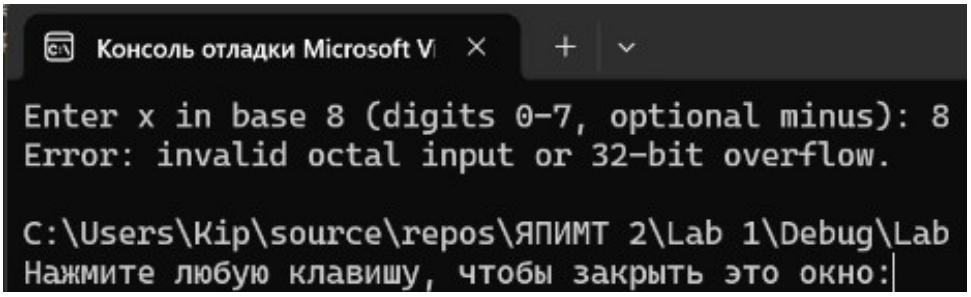
Тест № 4
Назначение: Проверка нулевого аргумента.
Ожидается: $x_{10} = 0$, $y_{10} = 1$.



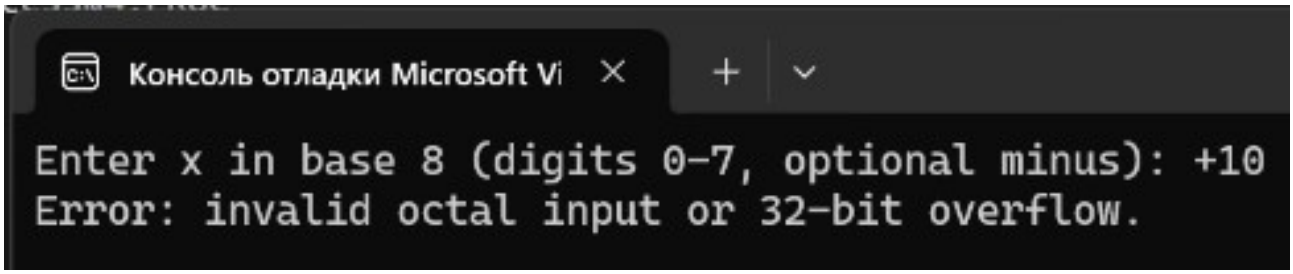
Тест № 5
Назначение: Проверка ввода с ведущими нулями.
Ожидается: $x_{10} = 15$, $y_{10} = 2446$.



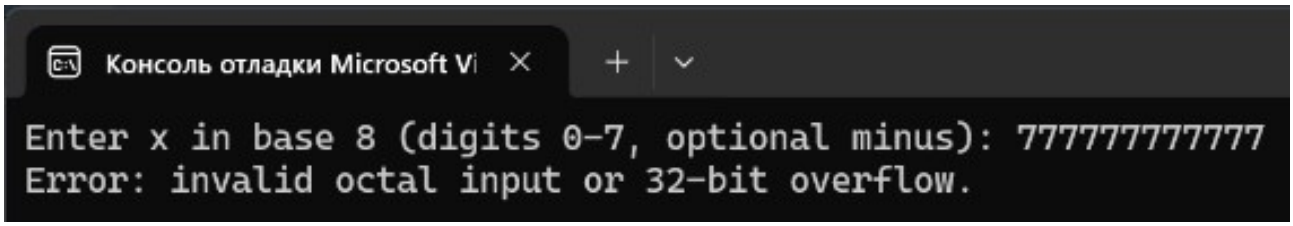
Тест № 6
Назначение: Проверка недопустимой восьмеричной цифры.
Ожидается: сообщение об ошибке.



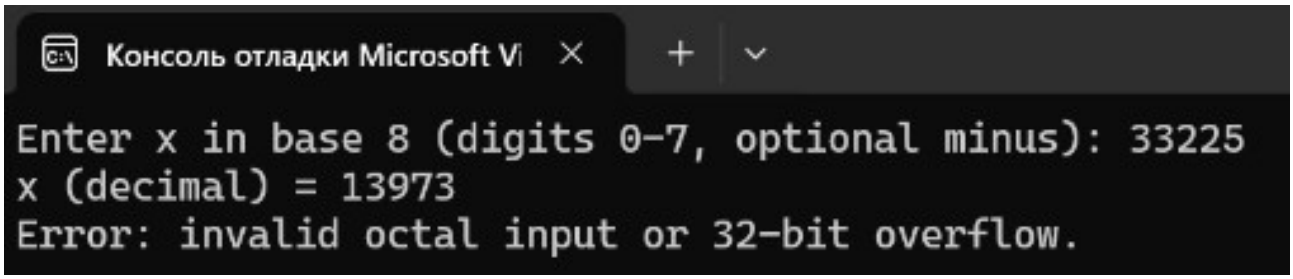
Тест № 7
Назначение: Проверка неподдерживаемого начального знака плюс.
Ожидается: сообщение об ошибке.



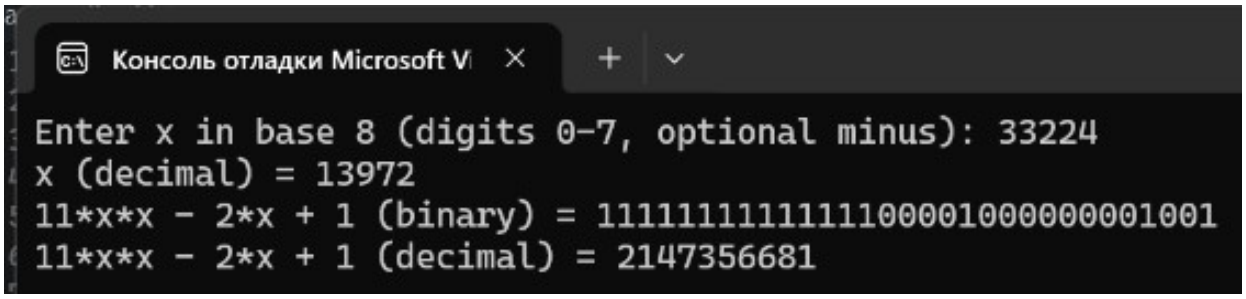
Тест № 8
Назначение: Проверка переполнения при преобразовании входной строки.
Ожидается: сообщение об ошибке.



Тест № 9
Назначение: Проверка переполнения при вычислении полинома.
Ожидается: сообщение об ошибке.



Тест № 10
Назначение: Проверка верхней границы допустимого диапазона.
Ожидается: $x_{10} = 13972$, $y_{10} = 2147356681$.



6 Код программы

```
.386
.MODEL FLAT, STDCALL
.STACK 4096

EXTERN GetStdHandle@4:PROC
EXTERN ReadConsoleA@20:PROC
EXTERN WriteConsoleA@20:PROC
EXTERN ExitProcess@4:PROC

.DATA
prompt      DB 'Enter x in base 8 (digits 0-7, optional minus): ',0
xLabel      DB 'x (decimal) = ',0
binLabel    DB '11*x*x - 2*x + 1 (binary) = ',0
decLabel    DB '11*x*x - 2*x + 1 (decimal) = ',0
errorText   DB 'Error: invalid octal input or 32-bit overflow.',13,10,0
newLine     DB 13,10,0
inputHandle DD ?
outputHandle DD ?
readCount   DD ?
written     DD ?
inputBuffer DB 64 DUP (?)
outputBuffer DB 34 DUP (?)
textAddress DD ?
textLength  DD ?
position    DD ?
negative    DD ?
digit       DD ?
value       DD ?
```

```
x           DD ?
square      DD ?
quadratic   DD ?
linear      DD ?
partial     DD ?
result      DD ?
number      DD ?
base        DD ?
quotient    DD ?
remainder   DD ?

.CODE
; EAX points to a zero-terminated string.
PrintText PROC
    MOV textAddress, EAX
    MOV textLength, 0
countChars:
    MOV EDX, textLength
    CMP BYTE PTR [EAX+EDX], 0
    JE writeText
    INC textLength
    JMP countChars
writeText:
    PUSH 0
    PUSH OFFSET written
    PUSH textLength
    PUSH textAddress
    PUSH outputHandle
    CALL WriteConsoleA@20
    RET
PrintText ENDP

; Read and convert a signed octal integer. Result is stored in x.
ReadOctal PROC
    PUSH 0
    PUSH OFFSET readCount
    PUSH 64
    PUSH OFFSET inputBuffer
    PUSH inputHandle
    CALL ReadConsoleA@20
    TEST EAX, EAX
    JZ inputError
    CMP readCount, 2
    JB inputError
    SUB readCount, 2
    MOV EDX, readCount
    CMP inputBuffer[EDX], 13
    JNE inputError
    CMP inputBuffer[EDX+1], 10
    JNE inputError
    MOV position, 0
    MOV negative, 0
    MOV value, 0
    CMP inputBuffer[0], '-'
    JNE checkDigits
    MOV negative, 1
    INC position
checkDigits:
    MOV EDX, position
    CMP EDX, readCount
    JAE inputError
readDigit:
    MOV EDX, position
    MOVZX EAX, inputBuffer[EDX]
    CMP EAX, '0'
    JB inputError
```

```
    CMP EAX, '7'
    JA inputError
    SUB EAX, '0'
    MOV digit, EAX
    MOV EAX, value
    IMUL EAX, 8
    JO inputError
    MOV value, EAX
    ADD EAX, digit
    JO inputError
    MOV value, EAX
    INC position
    MOV EDX, position
    CMP EDX, readCount
    JB readDigit
    CMP negative, 0
    JE storeX
    NEG value
storeX:
    MOV EAX, value
    MOV x, EAX
    RET
ReadOctal ENDP

; Convert number to base 2 or 10, filling the buffer from right to left.
PrintNumber PROC
    MOV position, 33
    MOV outputBuffer[33], 0
    MOV negative, 0
    MOV EAX, number
    CMP EAX, 0
    JGE magnitudeReady
    MOV negative, 1
    NEG EAX
magnitudeReady:
    MOV quotient, EAX
    CMP base, 2
    JNE nextDigit
nextBit:
    XOR EDX, EDX
    SHR EAX, 1
    ADC EDX, 0
    MOV quotient, EAX
    MOV remainder, EDX
    ADD EDX, '0'
    DEC position
    MOV ECX, position
    MOV outputBuffer[ECX], DL
    CMP EAX, 0
    JNE nextBit
    JMP checkSign
nextDigit:
    MOV EAX, quotient
    XOR EDX, EDX
    DIV base
    MOV quotient, EAX
    MOV remainder, EDX
    ADD EDX, '0'
    DEC position
    MOV ECX, position
    MOV outputBuffer[ECX], DL
    CMP quotient, 0
    JNE nextDigit
checkSign:
    CMP negative, 0
    JE printDigits
```

```
    DEC position
    MOV ECX, position
    MOV outputBuffer[ECX], '-'
printDigits:
    MOV EAX, OFFSET outputBuffer
    ADD EAX, position
    CALL PrintText
    MOV EAX, OFFSET newLine
    CALL PrintText
    RET
PrintNumber ENDP

MAIN PROC
    PUSH -10
    CALL GetStdHandle@4
    MOV inputHandle, EAX
    PUSH -11
    CALL GetStdHandle@4
    MOV outputHandle, EAX
    MOV EAX, OFFSET prompt
    CALL PrintText
    CALL ReadOctal

    MOV EAX, OFFSET xLabel
    CALL PrintText
    MOV EAX, x
    MOV number, EAX
    MOV base, 10
    CALL PrintNumber

    ; Store every intermediate value of 11*x*x - 2*x + 1.
    MOV EAX, x
    IMUL EAX, x
    JO inputError
    MOV square, EAX
    IMUL EAX, 11
    JO inputError
    MOV quadratic, EAX
    MOV EAX, x
    IMUL EAX, -2
    JO inputError
    MOV linear, EAX
    ADD EAX, quadratic
    JO inputError
    MOV partial, EAX
    ADD EAX, 1
    JO inputError
    MOV result, EAX

    MOV EAX, OFFSET binLabel
    CALL PrintText
    MOV EAX, result
    MOV number, EAX
    MOV base, 2
    CALL PrintNumber
    MOV EAX, OFFSET decLabel
    CALL PrintText
    MOV EAX, result
    MOV number, EAX
    MOV base, 10
    CALL PrintNumber
    PUSH 0
    CALL ExitProcess@4
MAIN ENDP

inputError:
```

```
    MOV EAX, OFFSET errorText
    CALL PrintText
    PUSH 1
    CALL ExitProcess@4
END MAIN
```

7 Вывод

Реализованы восьмеричный ввод, вычисление полинома и вывод в двоичной и десятичной системах. Промежуточные данные сохраняются в памяти. Тесты подтвердили правильность вычислений, обработки знака, ведущих нулей, некорректного ввода и переполнения.